Polygon Labs × Ignyte Submission — DeFa on Polygon Amoy

This document maps 1:1 to the Ignyte submission form. Each section corresponds to one form field so it can be copied straight in.

---

1. Team background and technical credentials

DeFa is built by the InvoiceMate team. Two-person build for this submission, with the surrounding InvoiceMate engineering pod supporting code review and QA outside the sprint:

- **Ibrahim Ansari — CEO.** Trade finance + fintech background. Owns product framing, regulatory framing, and the credit-workflow spec (KAM → CAD → CRO → Legal).
- **Hamza Anjum — CTO.** Full-stack + Solidity. Shipped the entire submission — `payfi_v1` contract set, Node/ethers backend, React lender UI, and the admin/PSP portal.

Technical credentials:

- Solidity + Foundry — 26 test files, ~17K LOC of test coverage on the `payfi_v1` contract set (unit, invariant, adversarial across 4 rounds, differential fuzzing against a Rust reference impl, gas-profile regressions, view-mutation consistency).
- Node.js 20 + Express + ethers v6 backend with SIWE (EIP-4361) auth, JWT session management, MongoDB persistence, and a live EVM event indexer.
- React 19 + Vite + Tailwind v4 + wagmi v2 + viem v2 + RainbowKit v2 for the LP UI. Existing Colosseum client (Vite React) forked to serve the PSP + admin portals.
- Deployment stack: Docker + docker-compose, containerised on Google Kubernetes Engine (GKE) with per-service Deployments (BE / lender-UI / admin-UI) fronted by nginx path-based routing.

The company shipping this: InvoiceMate Tech LTD — a regulated invoice financing operator with production infrastructure across UAE and South Africa. The DeFa submission is the on-chain credit-primitive layer of the same stack we run for banks and PSPs off-chain today. Existing engineering pod: 6 additional engineers across contracts, backend, frontend, and QA.

---

2. Problem statement and target market

Cross-border SME trade finance is roughly $2 trillion undersupplied globally (ADB Trade Finance Gaps Report, 2024). In the corridors we target — UAE ↔ Pakistan, UAE ↔ South Africa, Gulf ↔ East Africa — three specific frictions strand working capital:

1. **Settlement latency.** A PSP paying a supplier in Karachi against an invoice financed in Dubai waits 2–5 business days for SWIFT to clear. The SME is out of stock; the PSP carries the settlement risk on its balance sheet the whole time, and that cost gets priced back into every facility.
2. **FX drag on the drawdown leg.** When a facility is priced in USD, drawn in AED, and settled in PKR, the PSP eats the spread on every drawdown. Multiplied over hundreds of drawdowns per month, 200–400 bps of margin evaporates before it reaches the LP.
3. **No shared credit primitive.** Every bank runs its own KAM → CAD → CRO workflow in a separate silo. A PSP that's been underwritten by one bank has to redo the entire KYB and credit review to onboard with the next. There is no reusable on-chain primitive that says *"this PSP is underwritten, this pool is funded, here's the automated waterfall."*

Target market — a stack of four concentric segments:

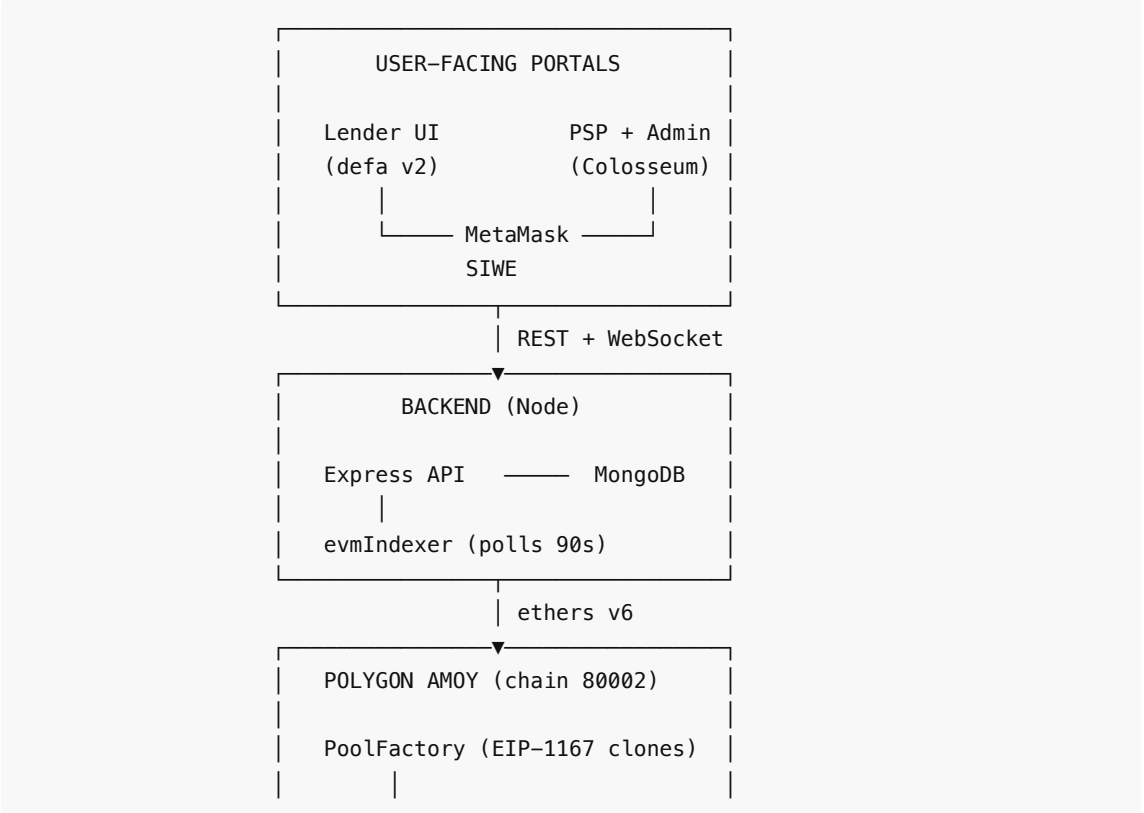
Segment	Size	How DeFa serves them
Cross-border Payment Service Providers (PSPs) in MENA + South Asia	~500 licensed PSPs across UAE / Pakistan / South Africa / East Africa	Pre-funding credit lines drawable in USDC, with sub-minute settlement to the supplier wallet.
SMEs financing receivables through those PSPs	~2M SMEs served collectively by the target PSPs	Receive USDC-denominated financing against verified customer orders, without waiting for SWIFT.
Institutional lenders wanting on-chain trade-finance yield	Family offices, private credit funds, DAO treasuries	Deposit USDC into risk-tiered pools, watch drawdowns and repayments settle in real time, claim yield atomically.
Retail liquidity in stablecoin-heavy regions	UAE (~30% crypto adoption), South Africa (top 5 globally)	Access via access-code onboarding, wallet-connect, and pools that surface the underlying real-economy activity.

Why Polygon: at SME ticket sizes ($5k–$50k per drawdown), Polygon PoS's sub-cent transaction cost leaves LP yield intact. On Ethereum L1 the gas cost would consume the yield; on Polygon it's rounding error. Polygon's EVM tooling maturity (Foundry, ethers, wagmi, RainbowKit all first-class) let us ship the full stack in a hackathon window without fighting the platform.

---

3. Technical architecture and approach on Polygon

Three-layer architecture



```
                    │   PoolContract ─── TreasuryReserve
                    │        │                          │
                    │        USDC (6-decimal)           │
                    └──────────────────────────────────┘
```

Full Mermaid diagrams (top-level graph, PSP → KAM → CAD → CRO sequence, contract class) live in `docs/ARCHITECTURE.md`.

Contract layer — `payfi_v1`

- **PoolFactory** — EIP-1167 minimal-proxy factory. `approvePsp(address)` gates who can operate a pool. `createPool(params)` clones a fresh `PoolContract` initialised with risk-envelope bounds.
- **PoolContract (per-facility)** — deposit / withdraw for LPs, `executeDrawdown(ref, receiver, amount, days)` for PSPs, `repay(ref)` waterfall (principal → utilization fee → penalty → protocol split → LP yield), `claimYield` + `claimPrincipal` for LPs, `declareDefault` fallback drawing from `TreasuryReserve`.
- **TreasuryReserve** — protocol fee sink + insurance reserve.
- **MockStablecoin** — 6-decimal USDC surrogate for testnet; swaps 1:1 to real USDC on Polygon PoS mainnet.

Security posture: OpenZeppelin v5 primitives (`AccessControl`, `ReentrancyGuard`, `Clones`, `SafeERC20`), `_disableInitializers` on the implementation, WAD fixed-point math with property-based invariant tests, ~17K LOC Foundry coverage across 26 test files. Details in `docs/CONTRACTS.md` and `SECURITY.md`.

Backend layer — Node / Express / ethers v6

- 18 route files: auth (SIWE + JWT), pool discovery, facility lifecycle, PSP KYB, KAM/CAD/CRO/Legal/on-chain-admin approvals, faucet, admin build-tx endpoints. Full API reference in `docs/API.md`.
- `poolServiceEvm` — reads live pool state and builds calldata for user wallets to sign.
- `walletAuthEvm` — SIWE (EIP-4361) with nonce store and domain binding.
- `evmIndexer` worker — polls `PoolCreated`, `Deposit`, `DrawdownExecuted`, `Repaid` events on a 90-second window and mirrors state into Mongo.
- `_withRetry` on all RPC calls — coalesces CALL_EXCEPTION, ECONNRESET, and missing-revert-data errors under burst load.

Frontend layer — two portals

- **DeFa Lender v2 UI** (`client/`) — React 19 + Vite + Tailwind v4 + wagmi v2 + viem v2 + RainbowKit v2. LPs browse tiered pools, deposit USDC (approve + deposit 2-step), watch utilisation live, redeem principal + yield.
- **PayMate PSP + admin portal** (`client-legacy/`) — the workflow surface. PSPs submit KYB and request facilities; KAM/CAD/CRO/Legal walk each facility through the approval chain; the on-chain-admin role signs the two-transaction pool bootstrap (`approvePsp` + `createPool`).

Deployed on Polygon Amoy (chain 80002)

Contract	Address
PoolFactory	`0xE02D8d3B14746E42c5D41a2CA805798D5A6E0F78`
MockUSD (6-decimal USDC surrogate)	`0x4e39880B43f9a83586a2aC75a01dff779Eb958c0`

TreasuryReserve	`0x03D21aFF05a94E2B87d1F55b2deE8F6f3fd9D9f8`

Four demo facilities are live on-chain right now under real names: **Wise Pay Partners**, **EFI Remitt**, **TransferGo Capital**, **Skrill Cross-Border**.

Test coverage

- **38 backend integration tests** across `e2eFlows` + `lifecycleFlows` + `onchainFlows`, running green on real Polygon Amoy.
- **~17K LOC Foundry coverage** on the contract set.

---

4. Launch roadmap and go-to-market strategy

Roadmap (5 phases)

Full detail in `docs/ROADMAP.md`; executive summary:

Phase	Timeline	Milestone	Volume target
Phase 1 — Sprint	Jul – Aug 2026	Ignyte submission · pilot facility with a live PSP partner in UAE ↔ Pakistan corridor · third-party audit engaged (OpenZeppelin or Trail of Bits)	1 pilot facility
Phase 2 — Q4 2026	Q4 2026	Migrate `MockStablecoin` → native USDC on Polygon PoS mainnet · `PoolFactory v2` with configurable per-pool envelopes · legal SPV wrapper per pool	3 live facilities
Phase 3 — H1 2027	H1 2027	3 live facilities across UAE ↔ Pakistan, UAE ↔ South Africa, Gulf ↔ Kenya · agent-driven KAM / CAD reviews live in production behind a human-approval fallback	**$10M cumulative drawdown volume**, $5M TVL
Phase 4 — H2 2027	H2 2027	Open `payfi_v1` to third-party PSPs (hosted-facility model) · multi-chain deployment (Polygon + Arc + Base) with CCTP-based liquidity balancing	**$50M cumulative drawdown volume**
Phase 5 — 2028+	2028 →	Public credit-scoring primitive (KYR scores as on-chain attestations) · trade finance as a stablecoin-native protocol	Open ecosystem

Go-to-market strategy

Our existing distribution surface is the launchpad, not a hypothesis:

1. **Warm-start liquidity from existing InvoiceMate LP relationships.** We already run pooled credit for institutional partners off-chain today; migrating those seats onto DeFa is a product upgrade, not a new sales cycle.
2. **PSP-side onboarding via existing bank partnerships.** Every bank we integrate with today already runs a KYB + credit workflow; adding an on-chain drawdown rail is an incremental route, not a rip-and-replace.

3. **Regulatory tailwind.** UAE PTSR + South Africa FSCA + DIFC Cat 3C already permit this activity — we are not waiting for regulation to catch up.
4. **Ecosystem partnerships.** Existing MoUs give us co-marketing and distribution surface across MENA + APAC.
5. **On Polygon specifically** — the Polygon Village / Aggregator surface gives us direct visibility to CDK-based ecosystem partners we haven't yet contacted, and Polygon's existing relationships with regulated fintechs in emerging markets (particularly Southeast Asia) accelerate PSP onboarding.

---

5. Revenue model and scalability plan

Revenue model

DeFa's revenue comes from four on-chain fee streams, all denominated in USDC and collected atomically as drawdowns and repayments settle. No off-chain invoicing, no delayed collection risk.

Fee	Rate	Payer	Splits to
Utilisation fee	30 bps/day on drawn principal	PSP (borrower)	80% LPs · 20% protocol (TreasuryReserve)
Commitment fee	5 bps/day on undrawn capacity	PSP	100% LPs (incentivises PSPs to draw efficiently)
Penalty fee	60 bps/day past grace period	PSP	60% LPs · 40% protocol (penalises delinquency, cushions defaults)
Origination fee	1% flat at facility creation	PSP	100% protocol

The protocol split accrues to `TreasuryReserve.depositImFees` on every `repay`. Sweep is admin-triggered via `/admin/build-tx/claim-protocol-fees`.

Illustrative unit economics — one active facility, $500k credit line, 90-day tenor, 60% average utilisation:

- Utilisation revenue: $500k × 60% × 30 bps × 90d = **$8,100** → $6,480 to LPs, $1,620 to protocol.
- Commitment revenue: $500k × 40% × 5 bps × 90d = **$900** → all to LPs.
- Assuming zero penalties: $9,000 total facility revenue, $7,380 to LPs (≈ 3.9% quarterly / ~15.6% APY on softCap), $1,620 to protocol per facility per quarter.

Scaling revenue linearly with facility count:

Phase	Facilities	Est. protocol revenue / year
Phase 3 (H1 2027)	3 facilities × $500k avg	~$19,000 protocol / yr
Phase 4 (H2 2027)	20 facilities × $750k avg	~$194,000 protocol / yr
Phase 5 (2028+)	100+ facilities × $1M avg	$1.6M+ protocol / yr

Volume targets ($10M in Phase 3 → $50M in Phase 4) drive proportional revenue.

Scalability plan

Contract layer scales linearly per facility, cheap:

- EIP-1167 minimal-proxy clones — deploying a new pool costs ~~420k gas (~~0.010 POL on Amoy, sub-cent on mainnet). No per-pool code duplication.
- Every operation (`deposit` , `executeDrawdown` , `repay` , `claimYield` , `claimPrincipal`) is O(1) w.r.t. facility count — no global state that grows with pool count.
- Full facility lifecycle (create → deposit → finalise → drawdown → repay → claim) costs ~0.032 POL on Amoy — see `docs/CONTRACTS.md` for the per-op gas table.

Backend scales horizontally:

- Stateless Node/Express services (`server/`) — containerised, each `POST` idempotent within the transaction envelope. Adding capacity = adding replicas behind the GKE load balancer.
- `evmIndexer` — currently one worker on a 90s poll window. Sharding strategy: split by pool-address hash mod N when pool count exceeds ~500, so each worker owns a slice. No cross-worker coordination needed (each pool is independent).
- Mongo Atlas — sharded by `pool` field on the hot collections (`drawdownstates` , `poolstates` , `poolevents`). At the target of 10,000 drawdowns / month = ~120k rows / year, well inside a single M20 cluster; the sharding path is designed but not yet required.

Frontend scales via CDN + static-first:

- Both React clients build to static bundles (`dist/`) served by nginx. Zero runtime compute per user — everything is either a static asset (page shell) or a proxied API call.
- Cloudflare / GCP CDN in front of the deploy handles edge caching. Time-to-first-byte remains constant regardless of user count.

Data pipeline scales by sharding on pool identity:

- Every event we care about (`PoolCreated` , `Deposit` , `DrawdownExecuted` , `Repaid`) is emitted with the pool address indexed. The indexer's block-range query filter is `{ address: [...trackedPools] }` — Polygon RPC handles multi-address filters natively, so 100 pools cost the same as 10 to poll per block-window.

Bottlenecks we've stress-tested for and mitigated:

- Polygon RPC burst limits — `_withRetry` wrapper on every RPC call catches CALL_EXCEPTION + coalesces missing revert data + ECONNRESET.
- Mongo write hotspot on `poolevents` — batch inserts per block window rather than per-event, indexed by `(pool, blockNumber)` .
- Frontend chunk size — code-split per route, main bundle < 500k gzipped; wallet + charts loaded lazily.

---

6. MVP / Prototype

Live, working, verifiable on-chain:

Layer	Live URL	Notes
Lender UI	https://defa-polygon-hackathon.invoicemate.net/	Access code `123456` for signup
PSP + Admin UI	https://defa-polygon-hackathon-admin.invoicemate.net/	Login creds in the how-to-test section below

Backend API	https://defa-polygon-hackathon.invoicemate.net/api	Full route reference in `docs/API.md`
Contracts	Polygon Amoy (chain 80002)	Addresses in Section 3 above

How to test — every role, with credentials:

Full walkthrough is on the repo landing page (`README.md` → "How to test the live demo"). Quick summary:

Role	Login	What they do
PSP / Borrower	`psp@demo.invoicemate.net` / `demo123`	Submit KYB, view credit line, request facility, execute drawdown against real external orders
KAM	`kam@maildrop.cc` / `admin123`	First-tier facility approval
CAD	`cad@maildrop.cc` / `admin123`	Credit + documentation approval
CRO	`cro@maildrop.cc` / `admin123`	Risk approval, upload credit memo PDF, set risk params
Onchain Admin	Connect wallet (see below)	Sign `factory.approvePsp` + `factory.createPool` on chain
LP / Lender	Access code `123456` + connect wallet	Browse pools, deposit USDC, claim yield

Test wallet (Polygon Amoy, pre-funded with 1M MockUSD + on the on-chain admin allowlist):

```
Private key: 0x692fb6f9b2c22e3d2ad4e0434f22f41617fb65ce1ac89146da2ae21b58443ce9
Address:     0x0b9dDfcdB31aEf5Cde26d0E6DbAc6917B6849f05
```

(Deliberately published for the hackathon — will be rotated post-submission. Treated as a burner wallet and pre-authorised in `.env.production.example`.)

Full lifecycle in one 11-step flow (verifiable on-chain):

```
1.  PSP signs up             → KYB submission created
2.  PSP requests facility    → Facility.status = KAM_REVIEW
3.  KAM approves             → CAD_REVIEW
4.  CAD approves             → CRO_REVIEW
5.  CRO approves + terms     → AWAITING_POOL_INIT
6.  Onchain admin signs      → factory.approvePsp + factory.createPool
7.  evmIndexer picks up      → pool visible on /api/pools within 90s
8.  LP deposits              → approve + deposit (2-step)
9.  PSP executes drawdown    → server signs AGENT2, USDC to receiver
10. PSP repays               → principal + utilisation fee + penalty
11. LP redeems               → claimYield + claimPrincipal (2-step)
```

Repository: https://github.com/Mate-Sol/The-Smart-Commerce-Infrastructure-Challenge-Polygon-Labs-

Additional supporting docs in the repo:

- `docs/ARCHITECTURE.md` — Mermaid architecture diagrams

- `docs/CONTRACTS.md` — one-page contract reference
- `docs/API.md` — backend API reference
- `docs/ROADMAP.md` — 5-phase launch roadmap
- `docs/LOCAL_E2E.md` — local dev walkthrough
- `docs/USER_FLOW.md` — end-user narrative
- `docs/REPAYMENT_LOGIC.md` — waterfall math reference
- `SECURITY.md` — contract security posture, audit plan
- `server/TESTING.md` — 38-test integration suite documentation

Video demonstration: `<paste YouTube unlisted link when uploaded>`

Pitch deck: `<paste Google Slides / PDF link>`